

part D. Objects

Object Types

When you create an object literal with `{...}` syntax, TypeScript will consider it to be a new object type, or type shape, based on its properties. That object type will have the same property names and primitive types as the object's values. Accessing properties of the value can be done with either `value.member` or the equivalent `value['member']` syntax.

TypeScript understands that the following `poet` variable's type is that of an object with two properties: `born`, of type `number`, and `name`, of type `string`. Accessing those members would be allowed, but attempting to access any other member name would cause a type error for that name not existing:

```
const poet = {
  born: 1935,
  name: "Mary Oliver",
};

poet['born']; // Type: number
poet.name;   // Type: string

poet.end;
//    ~~~
// Error: Property 'end' does not exist on
// type '{ born: number; name: string; }'.
```

Object types are a core concept for how TypeScript understands JavaScript code. Every value other than `null` and `undefined` has a set of members in its backing type shape, and so TypeScript must understand the object type for every value in order to type check it.

Declaring Object Types

Inferring types directly from existing objects is all fine and good, but eventually you'll want to be able to declare the

type of an object explicitly. You'll need a way to describe an object shape separately from objects that satisfy it.

Object types may be described using a syntax that looks similar to object literals but with types instead of values for fields. It's the same syntax that TypeScript shows in error messages about type assignability.

This `poetLater` variable is the same type from before with `name: string` and `born: number`:

```
let poetLater: {  
  born: number;  
  name: string;  
};  
  
// Ok  
poetLater = {  
  born: 1935,  
  name: "Mary Oliver",  
};  
  
poetLater = "Sappho";  
// Error: Type 'string' is not assignable to  
// type '{ born: number; name: string; }'
```

Aliased Object Types

Constantly writing out object types like `{ born: number; name: string; }` would get tiresome rather quickly. It's more common to use type aliases to assign each type shape a name.

The previous code snippet could be rewritten with a `type Poet`, which comes with the added benefit of making TypeScript's assignability error message a little more direct and readable:

```
type Poet = {  
  born: number;  
  name: string;  
};  
  
let poetLater: Poet;  
  
// Ok  
poetLater = {
```

```
    born: 1935,
    name: "Sara Teasdale",
  };

poetLater = "Emily Dickinson";
// Error: Type 'string' is not assignable to type 'Poet'.
```

I bring these object types up now because understanding how TypeScript interprets object literals is an important part of learning about TypeScript’s type system. These concepts will continue to be important once we switch over to features in the next section of this material.

Structural Typing

TypeScript’s type system is *structurally typed*: meaning any value that happens to satisfy a type is allowed to be used as a value of that type. In other words, when you declare that a parameter or variable is of a particular object type, you’re telling TypeScript that whatever object(s) you use, they need to have those properties.

The following `WithFirstName` and `WithLastName` aliased object types both only declare a single member of type `string`. The `hasBoth` variable just so happens to have both of them—even though it wasn’t declared as such explicitly—so it can be provided to variables that are declared as either of the two aliased object types:

```
type WithFirstName = {
  firstName: string;
};

type WithLastName = {
  lastName: string;
};

const hasBoth = {
  firstName: "Lucille",
  lastName: "Clifton",
};

// Ok: `hasBoth` contains a `firstName` property of type `string`
let withFirstName: WithFirstName = hasBoth;
```

```
// Ok: `hasBoth` contains a `lastName` property of type `string`  
let withLastName: WithLastName = hasBoth;
```

Structural typing is not the same as *duck typing*, which comes from the phrase “If it looks like a duck and quacks like a duck, it’s probably a duck.”

- Structural typing is when there is a static system checking the type—in TypeScript’s case, the type checker.
- Duck typing is when nothing checks object types until they’re used at runtime.

In summary: *JavaScript* is *duck typed* whereas *TypeScript* is *structurally typed*.

Usage Checking

When providing a value to a location annotated with an object type, TypeScript will check that the value is assignable to that object type. To start, the value must have the object type’s required properties. If any member required on the object type is missing in the object, TypeScript will issue a type error.

The following `FirstAndLastNames` aliased object type requires that both the `first` and `last` properties exist. An object containing both of those is allowed to be used in a variable declared to be of type `FirstAndLastNames`, but an object without them is not:

```
type FirstAndLastNames = {  
  first: string;  
  last: string;  
};  
  
// Ok  
const hasBoth: FirstAndLastNames = {  
  first: "Sarojini",  
  last: "Naidu",  
};  
  
const hasOnlyOne: FirstAndLastNames = {  
  first: "Sappho"  
};  
// Property 'last' is missing in type '{ first: string; }'  
// but required in type 'FirstAndLastNames'.
```

Mismatched types between the two are not allowed either. Object types specify both the names of required properties and the types those properties are expected to be. If an object's property doesn't match, TypeScript will report a type error.

The following `TimeRange` type expects the `start` member to be of type `Date`. The `hasStartString` object is causing a type error because its `start` is type `string` instead:

```
type TimeRange = {
  start: Date;
};

const hasStartString: TimeRange = {
  start: "1879-02-13",
  // Error: Type 'string' is not assignable to type 'Date'.
};
```

Excess Property Checking

TypeScript will report a type error if a variable is declared with an object type and its initial value has more fields than its type describes. Therefore, declaring a variable to be of an object type is a way of getting the type checker to make sure it has only the expected fields on that type.

The following `poetMatch` variable has exactly the fields described in the object type aliased by `Poet`, while `extraProperty` causes a type error for having an extra property:

```
type Poet = {
  born: number;
  name: string;
}

// Ok: all fields match what's expected in Poet
const poetMatch: Poet = {
  born: 1928,
  name: "Maya Angelou"
};

const extraProperty: Poet = {
  activity: "walking",
  born: 1935,
  name: "Mary Oliver",
```

```
};  
// Error: Type '{ activity: string; born: number; name: string; }'  
// is not assignable to type 'Poet'.  
//   Object literal may only specify known properties,  
//   and 'activity' does not exist in type 'Poet'.
```

Note that excess property checks only trigger for object literals being created in locations that are declared to be an object type. Providing an existing object literal bypasses excess property checks.

This `extraPropertyButOk` variable does not trigger a type error with the previous example's `Poet` type because its initial value happens to structurally match `Poet`:

```
const existingObject = {  
  activity: "walking",  
  born: 1935,  
  name: "Mary Oliver",  
};  
  
const extraPropertyButOk: Poet = existingObject; // Ok
```

Excess property checks will trigger anywhere a new object is being created in a location that expects it to match an object type—which as you'll see in later parts includes array members, class fields, and function parameters. Banning excess properties is another way TypeScript helps make sure your code is clean and does what you expect. Excess properties not declared in their object types are often either mistyped property names or unused code.

Nested Object Types

As JavaScript objects can be nested as members of other objects, TypeScript's object types must be able to represent nested object types in the type system. The syntax to do so is the same as before but with a `{ ... }` object type instead of a primitive name.

`Poem` type is declared to be an object whose `author` property has `firstName: string` and `lastName: string`. The `poemMatch` variable is assignable to `Poem` because it matches that structure, while `poemMismatch` is not because

its `author` property includes `name` instead of `firstName` and `lastName`:

```
type Poem = {
  author: {
    firstName: string;
    lastName: string;
  };
  name: string;
};

// Ok
const poemMatch: Poem = {
  author: {
    firstName: "Sylvia",
    lastName: "Plath",
  },
  name: "Lady Lazarus",
};

const poemMismatch: Poem = {
  author: {
    name: "Sylvia Plath",
  },
  // Error: Type '{ name: string; }' is not assignable
  // to type '{ firstName: string; lastName: string; }'.
  //   Object literal may only specify known properties, and 'name'
  //   does not exist in type '{ firstName: string; lastName: string; }'.
  name: "Tulips",
};
```

Another way of writing the `type Poem` would be to extract out the `author` property's shape into its own aliased object type, `Author`. Extracting out nested types into their own type aliases also helps TypeScript give more informative type error messages. In this case, it can say '`Author`' instead of '`{ firstName: string; lastName: string; }`':

```
type Author = {
  firstName: string;
  lastName: string;
};

type Poem = {
  author: Author;
  name: string;
};
```



```
const poemMismatch: Poem = {
  author: {
    name: "Sylvia Plath",
  },
  // Error: Type '{ name: string; }' is not assignable to type 'Author'.
  //      Object literal may only specify known properties,
  //      and 'name' does not exist in type 'Author'.
  name: "Tulips",
};
```

You'll see in later part how object type members can be other types such as arrays and functions.

Optional Properties

Object type properties don't all have to be required in the object. You can include a `?` before the `:` in a type property's type annotation to indicate that it's an optional property.

This `Book` type requires only a `pages` property and optionally allows an `author`. Objects adhering to it may provide `author` or leave it out as long as they provide `pages`:

```
type Book = {
  author?: string;
  pages: number;
};

// Ok
const ok: Book = {
  author: "Rita Dove",
  pages: 80,
};

const missing: Book = {
  author: "Rita Dove",
};
// Error: Property 'pages' is missing in type
// '{ author: string; }' but required in type 'Book'.
```

Keep in mind there is a difference between optional properties and properties whose type happens to include `undefined` in a type union. A property declared as optional with `?` is allowed to not exist. A property declared as required and `| undefined` must exist, even if the value is `undefined`.

The `editor` property in the following `Writers` type may be skipped in declaring variables because it has a `?` in its declaration. The `author` property does not have a `?`, so it must exist, even if its value is just `undefined`:

```
type Writers = {
  author: string | undefined;
  editor?: string;
};

// Ok: author is provided as undefined
const hasRequired: Writers = {
  author: undefined,
};

const missingRequired: Writers = {};
//      ~~~~~
// Error: Property 'author' is missing in type
// '{}' but required in type 'Writers'.
```

Unions of Object Types

It is reasonable in TypeScript code to want to be able to describe a type that can be one or more different object types that have slightly different properties. Furthermore, your code might want to be able to type narrow between those object types based on the value of a property.

Inferred Object-Type Unions

If a variable is given an initial value that could be one of multiple object types, TypeScript will infer its type to be a union of object types. That union type will have a constituent for each of the possible object shapes. Each of the possible properties on the type will be present in each of those constituents, though they'll be `?` optional types on any type that doesn't have an initial value for them.

This `poem` value always has a `name` property of type `string`, and may or may not have `pages` and `rhymes` properties:

```
const poem = Math.random() > 0.5
  ? { name: "The Double Image", pages: 7 }
  : { name: "Her Kind", rhymes: true };
```

```
// Type:
// {
//   name: string;
//   pages: number;
//   rhymes?: undefined;
// }
// |
// {
//   name: string;
//   pages?: undefined;
//   rhymes: boolean;
// }

poem.name; // string
poem.pages; // number | undefined
poem.rhymes; // boolean | undefined
```

Explicit Object-Type Unions

Alternately, you can be more explicit about your object types by being explicit with your own union of object types. Doing so requires writing a bit more code but comes with the advantage of giving you more control over your object types. Most notably, if a value's type is a union of object types, TypeScript's type system will only allow access to properties that exist on all of those union types.

This version of the previous `poem` variable is explicitly typed to be a union type that always has the `name` property along with either `pages` or `rhymes`. Accessing `name` is allowed because it always exists, but `pages` and `rhymes` aren't guaranteed to exist:

```
type PoemWithPages = {
  name: string;
  pages: number;
};

type PoemWithRhymes = {
  name: string;
  rhymes: boolean;
};

type Poem = PoemWithPages | PoemWithRhymes;

const poem: Poem = Math.random() > 0.5
  ? { name: "The Double Image", pages: 7 }
  : { name: "The Double Image", rhymes: true };
```

```

: { name: "Her Kind", rhymes: true };

poem.name; // Ok

poem.pages;
// ~~~~~
// Property 'pages' does not exist on type 'Poem'.
//   Property 'pages' does not exist on type 'PoemWithRhymes'.

poem.rhymes;
// ~~~~~
// Property 'rhymes' does not exist on type 'Poem'.
//   Property 'rhymes' does not exist on type 'PoemWithPages'.

```

Restricting access to potentially nonexistent members of objects can be a good thing for code safety. If a value might be one of multiple types, properties that don't exist on all of those types aren't guaranteed to exist on the object.

Just as how unions of literal and/or primitive types must be type narrowed to access properties that don't exist on all type constituents, you'll need to narrow those object type unions.

Narrowing Object Types

If the type checker sees that an area of code can only be run if a union typed value contains a certain property, it will narrow the value's type to only the constituents that contain that property. In other words, TypeScript's type narrowing will apply to objects if you check their shape in code.

Continuing the explicitly typed `poem` example, check whether `"pages" in poem` acts as a type guard for TypeScript to indicate that it is a `PoemWithPages`. If `poem` is not a `PoemWithPages`, then it must be a `PoemWithRhymes`:

```

if ("pages" in poem) {
    poem.pages; // Ok: poem is narrowed to PoemWithPages
} else {
    poem.rhymes; // Ok: poem is narrowed to PoemWithRhymes
}

```

Note that TypeScript won't allow truthiness existence checks like `if (poem.pages)`. Attempting to access a property of

an object that might not exist is considered a type error, even if used in a way that seems to behave like a type guard:

```
if (poem.pages) { /* ... */ }  
//      ~~~~~  
// Property 'pages' does not exist on type 'Poem'.  
//   Property 'pages' does not exist on type 'PoemWithRhymes'.
```

Discriminated Unions

Another popular form of union typed objects in JavaScript and TypeScript is to have a property on the object indicate what shape the object is. This kind of type shape is called a *discriminated union*, and the property whose value indicates the object's type is a *discriminant*. TypeScript is able to perform type narrowing for code that type guards on discriminant properties.

For example, this `Poem` type describes an object that can be either a new `PoemWithPages` type or a new `PoemWithRhymes` type, and the `type` property indicates which one. If `poem.type` is `"pages"`, then TypeScript is able to infer that the type of `poem` must be `PoemWithPages`. Without that type narrowing, neither property is guaranteed to exist on the value:

```
type PoemWithPages = {  
  name: string;  
  pages: number;  
  type: 'pages';  
};  
  
type PoemWithRhymes = {  
  name: string;  
  rhymes: boolean;  
  type: 'rhymes';  
};  
  
type Poem = PoemWithPages | PoemWithRhymes;  
  
const poem: Poem = Math.random() > 0.5  
  ? { name: "The Double Image", pages: 7, type: "pages" }  
  : { name: "Her Kind", rhymes: true, type: "rhymes" };  
  
if (poem.type === "pages") {  
  console.log(`It's got pages: ${poem.pages}`); // Ok
```

```

} else {
    console.log(`It rhymes: ${poem.rhymes}`);
}

poem.type; // Type: 'pages' | 'rhymes'

poem.pages;
// ~~~~~
// Error: Property 'pages' does not exist on type 'Poem'.
// Property 'pages' does not exist on type 'PoemWithRhymes'.

```

Discriminated unions are my favorite feature in TypeScript because they beautifully combine a common elegant JavaScript pattern with TypeScript’s type narrowing. and its associated projects will show more around using discriminated unions for generic data operations.

Intersection Types

TypeScript’s `|` union types represent the type of a value that could be one of two or more different types. Just as JavaScript’s runtime `|` operator acts as a counterpart to its `&` operator, TypeScript allows representing a type that is multiple types at the same time: an `&` *intersection type*. Intersection types are typically used with aliased object types to create a new type that combines multiple existing object types.

The following `Artwork` and `Writing` types are used to form a combined `WrittenArt` type that has the properties `genre`, `name`, and `pages`:

```

type Artwork = {
    genre: string;
    name: string;
};

type Writing = {
    pages: number;
    name: string;
};

type WrittenArt = Artwork & Writing;
// Equivalent to:
// {
//   genre: string;

```

```
//   name: string;  
//   pages: number;  
// }
```

Intersection types can be combined with union types, which is sometimes useful to describe discriminated unions in one type.

This `ShortPoem` type always has an `author` property, then is also a discriminated union on a `type` property:

```
type ShortPoem = { author: string } & (  
  | { kigo: string; type: "haiku"; }  
  | { meter: number; type: "villanelle"; }  
);  
  
// Ok  
const morningGlory: ShortPoem = {  
  author: "Fukuda Chiyo-ni",  
  kigo: "Morning Glory",  
  type: "haiku",  
};  
  
const oneArt: ShortPoem = {  
  author: "Elizabeth Bishop",  
  type: "villanelle",  
};  
// Error: Type '{ author: string; type: "villanelle"; }'  
// is not assignable to type 'ShortPoem'.  
//   Type '{ author: string; type: "villanelle"; }' is not assignable to  
//   type '{ author: string; } & { meter: number; type: "villanelle"; }'.  
//     Property 'meter' is missing in type '{ author: string; type: "villanelle"; }'  
//     but required in type '{ meter: number; type: "villanelle"; }'.
```

Dangers of Intersection Types

Intersection types are a useful concept, but it's easy to use them in ways that confuse either yourself or the TypeScript compiler. I recommend trying to keep code as simple as possible when using them.

Long assignability errors

Assignability error messages from TypeScript get much harder to read when you create complex intersection types, such as one combined with a union type. This will be a common theme with TypeScript's type system (and typed pro-

gramming languages in general): the more complex you get, the harder it will be to understand messages from the type checker.

In the case of the previous code snippet's `ShortPoem`, it would be much more readable to split the type into a series of aliased object types to allow TypeScript to print those names:

```
type ShortPoemBase = { author: string };
type Haiku = ShortPoemBase & { kigo: string; type: "haiku" };
type Villanelle = ShortPoemBase & { meter: number; type: "villanelle" };
type ShortPoem = Haiku | Villanelle;

const oneArt: ShortPoem = {
  author: "Elizabeth Bishop",
  type: "villanelle",
};
// Type '{ author: string; type: "villanelle"; }'
// is not assignable to type 'ShortPoem'.
//   Type '{ author: string; type: "villanelle"; }'
//   is not assignable to type 'Villanelle'.
//     Property 'meter' is missing in type
//     '{ author: string; type: "villanelle"; }'
//     but required in type '{ meter: number; type: "villanelle"; }'.
```

never

Intersection types are also easy to misuse and create an impossible type with. Primitive types cannot be joined together as constituents in an intersection type because it's impossible for a value to be multiple primitives at the same time. Trying to `&` two primitive types together will result in the *never* type, represented by the keyword `never`:

```
type NotPossible = number & string;
// Type: never
```

The `never` keyword and type is what programming languages refer to as a *bottom type*, or empty type. A bottom type is one that can have no possible values and can't be reached. No types can be provided to a location whose type is a bottom type:

```
let notNumber: NotPossible = 0;
// ~~~~~
```



```
// Error: Type 'number' is not assignable to type 'never'.

let notString: never = "";
// ~~~~~
// Error: Type 'string' is not assignable to type 'never'.
```

Most TypeScript projects rarely—if ever—use the `never` type. It comes up once in a while to represent impossible states in code. Most of the time, though, it’s likely to be a mistake from misusing intersection types. in .

In this part, you expanded your grasp of the TypeScript type system to be able to work with objects:

- How TypeScript interprets types from object type literals
- Describing object literal types, including nested and optional properties
- Declaring, inferring, and type narrowing with unions of object literal types
- Discriminated unions and discriminants
- Combining object types together with intersection types